

Bonus target: tomlc99 (TOML)

This is the secondary report. I tested a second target, TOML, using the same pipeline and the same fuzzer/code, with no changes needed to support it. The design details (how crashes are told apart from valid rejections, the proxy signal, the two-pass testing approach, how duplicate crashes are grouped) are the same as in the primary report and aren't repeated here.

Target: [tomlc99](#) @ 29076df.

Grammar: grammars-v4 toml/{TomlParser, TomlLexer}.g4 @ e1c222f.

Result: 5 iterations, 2,500 inputs, **4 crash signatures**, **1 confirmed root cause**, \$2.76 of a \$5 budget.

Why I added a second target

The JSON run found nothing, and a single negative result is hard to read on its own. It could mean parson is genuinely solid, or it could mean my pipeline just can't find bugs at all. Testing a second library answers these two questions at once: does the same pipeline work without changes on a different target, and does a less-hardened parser behave differently? Both answers turned out to be yes. Nothing in `outcomes.py`, `runner.py`, `triage.py`, `coverage.py`, or `campaign.py` needed to change, and TOML produced a real bug.

Findings

A real bug, found twice

Before running the agentic loop, I tested the library by hand and found `toml.c` has no limit on how deeply it will nest arrays or tables. `parson`, by contrast, has an explicit cap (`MAX_NESTING 2048`) and rejects anything past it cleanly. `tomlc99` has no such check. It just keeps recursing until the program crashes.

I built an input nested 50,000 levels deep, and it overflowed the stack. The exact point where it breaks isn't fixed. It falls somewhere between roughly 23,000 and 27,000, depending on the shape of the input and small differences between runs (`grammar/toml-tomlc99/ADAPTATIONS.md`).

I gave the loop this finding as a starting point, and it went on to generate its own deeply nested inputs that hit the same crash on its own, using its own strategies. Finding it by hand only proves the bug exists. Finding it through the loop proves that the depth signal points toward bugs, instead of the loop just happening to trip over one.

Four signatures, one bug

The triage pipeline reported four separate crash signatures. But reading the actual stack traces, all four are the same thing: `AddressSanitizer: stack-overflow`. They only look different because of what the program happened to be doing right when it ran out of stack space: allocating memory during array growth, duplicating a key, or in one case nothing at all, because the crash unwinder itself ran out of stack. Strip that away, and the real call chain underneath all four is thousands of frames of the same two functions calling themselves with no depth limit:

```
parse_array -> parse_array -> parse_array -> ...      (array nesting)
parse_inline_table -> parse_keyval -> parse_inline_table -> ... (inline-table nesting)
```

The primary report admitted that my crash-grouping method had never been tested on a real crash, only a toy one. Here it finally was, and it exposed a real weakness. My method groups crashes by hashing the last few stack frames, but for a stack-overflow bug those frames depend on timing, not on what caused the crash, so one real bug can show up as several different-looking signatures. The loop noticed this itself in iteration 3. Its own notes flagged the jump from 3 to 4 signatures as "the same stack-overflow bug family," and redirected its budget instead of re-hunting a bug already in hand. So, the honest count is one confirmed bug: unbounded recursion, reachable through at least two code paths, that showed up as four raw signatures. I re-ran all four saved crash inputs by hand against the pinned build, five times each. All twenty runs

crashed, but the signature itself is not stable: one byte-identical input hashed to four different IDs across those runs, including a fifth ID that never appeared during the campaign at all. The four signatures were never four behaviors.

The reproducers are large, and that's expected, not a flaw

Three of the four saved crash inputs are not fully minimized and range from 53 to 137 KB, because the exact crash depth shifts by roughly a thousand levels from one run to the next, the shrinker's smaller test candidates sometimes don't crash at all. Rather than report a smaller input that might not actually reproduce the bug, the shrinker keeps the original, larger one.

How the generator changed over five rounds

Iteration	Acceptance	Max depth	Crashes	What changed
0	6.2%	25,704	0	first version, seeded from the grammar and measured gaps
1	43.0%	33,701	3	moved deep nesting into its own strategy
2	39.4%	31,381	4	found the crash, then reduced how aggressively it nested
3	28.8%	33,347	1 (re-hit)	recognized it was one bug family, shifted focus to acceptance
4	48.0%	21,594	0	worked on string content and recovering acceptance rate

Iteration 0's 6.2% acceptance rate is a textbook example of the exact failure this whole approach is meant to catch. The first generator added an unlimited nesting draw inside every single document, so roughly 40% of otherwise normal documents got dragged down and rejected because of one runaway field. This is the kind of "generator that's mostly getting rejected" case the assignment specifically asks about. From the acceptance number alone, iteration 1 figured out the problem, gave deep nesting its own separate strategy instead of mixing it into every document, and acceptance jumped to 43% in one round. Reviewing the code after the run, I found the harness was discarding tom99's error messages, so the model never saw a single parser rejection for this target and had nothing but the acceptance rate to reason from.

After that, the loop did what I'd hoped it would. Iteration 2 found the crash and immediately backed off how hard it pushed nesting, since a bug that's already reproducible doesn't need more pressure behind it. Iterations 3 and 4 spent the rest of the budget improving acceptance and testing string content at the byte level, following the note in ADAPTATIONS.md that tom99 already handles numbers carefully, so that wasn't worth spending more budget on.

Challenges specific to this target

A crash point that moves makes "found the bug" a probability, not a fact. parson's nesting limit was exact and identical every time I tested it. tom99's crash depends on how the stack happens to be laid out on a given run, so saying "it crashes at depth X" is only true for that one run, not a fixed property of the bug. The pipeline handles this, since it verifies before reporting and refuses to over-minimize, but it does change what the evidence actually proves.

The crash-grouping method really does overcount. I deliberately didn't fix this here, because adjusting the grouping logic after already knowing the answer would be circular reasoning. The honest fix for later would be a second grouping pass that also checks whether the sanitizer error type matches and whether one function dominates the stack, which would correctly merge all four signatures into one.

Artifacts under tom99-tom99: grammar/, target/, strategies/, logs/, crashes/.